

1. CORE IDEA

The Problem

Publishing solved coding problems to GitHub manually is tedious. It requires: creating folders, copying/renaming files, writing README documentation, placing screenshots, initializing/managing git branches, making commits, and opening pull requests.

The Solution

One VS Code command → Complete publishing workflow.

Write Code → Run "GitGo: Publish Solution" → Done

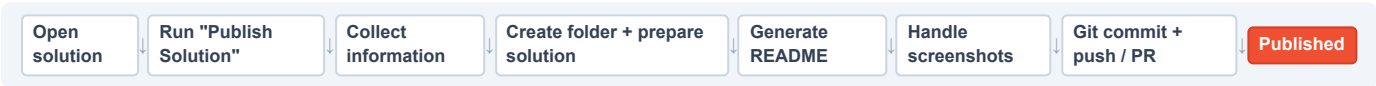
30-Second Interview Answer

"GitGo is a VS Code extension built in TypeScript that automates the entire process of saving and publishing coding solutions to GitHub. With a single command, it handles folder creation, template-based README generation, file-renaming, screenshot handling, and Git operations (including branch creation and PR generation) as a single rollback-safe operation."

2. MAIN TECHNOLOGIES

Technology	What we use it for
TypeScript	Write the extension with static type safety, reducing runtime errors across stages.
VS Code Extension API	Add the GitGo commands directly to VS Code host, register commands, and interact with the editor context.
Node.js	Run the extension logic, handle the filesystem, and execute child processes for CLI automation.
Git CLI	Automate Git repository operations (status check, branch switching, committing, pushing, PR creation).
esbuild	Bundle the extension files into a fast, single-file distribution for distribution.

3. PUBLISHING FLOW



4. MAIN COMPONENTS & RESPONSIBILITIES

Architectural Principle: "Each part has one responsibility instead of one huge function doing everything."

Entry Layer

VS Code Extension Host
Command activation & registration

VS Code Command
"User starts GitGo"

Application Layer

Publish Pipeline Orchestrator
`controls the workflow`

Coordinates all stages in order. Ensures rollback safety if any step fails.

Pipeline Stages

FolderCreator
`creates the problem folder`

ReadmeGenerator
`creates README`

GitService
`handles Git operations`

5. PUBLISH CONTEXT

Instead of passing many separate arguments, GitGo aggregates pipeline state into one object:

`PublishContext`

[Problem • Language • Folder • Author • Repo • Push Mode]

Why this matters:

"PublishContext gives the different stages a common set of information to work with."

7. GIT AUTOMATION

Normal Push: Files → Commit → Push → Done

PR Mode: Files → New Branch → Commit → Push → PR

DefaultBranchDetector: Queries the repository configuration instead of assuming the default branch is main or master.

9. MEMORY & SETTINGS

GitGo remembers state to avoid repetitive user inputs:

- Repository path
- Author details
- Last used parent folder

11. RAPID-FIRE INTERVIEW QA

1. What is GitGo?

A VS Code extension that automates publishing coding-problem solutions to GitHub with one command.

3. What is the Publish Pipeline Orchestrator?

The component that controls the complete publishing workflow and executes each stage in the correct order.

5. What is PublishContext?

A shared object containing the information required by the different publishing stages.

7. Why call it transactional?

Because the entire publishing workflow is treated as one operation: either it completes successfully or changes are rolled back.

9. How does GitGo know the default branch?

The DefaultBranchDetector checks the repository instead of assuming `main` or `master`.

11. What does GitGo automate?

File creation, naming, README generation, screenshots, Git operations, branches and PR workflow.

6. TRANSACTIONAL PIPELINE

Problem: What if Folder is created ✓, README is created ✓, but Git push fails X? The repo is left half-finished.

Solution: GitGo treats publishing as one **Transactional/Atomic** operation. If a stage fails, it executes rollbacks in reverse order.

Start → Execute Stages → Success? Finish | Fail? Rollback previous changes

8. README & FILE AUTOMATION

- **File Renaming:** Standardizes files according to resolved target naming conventions.
- **ReadmeGenerator:** Uses specific templates for LeetCode (difficulty, runtime) vs Normal problems.
- **ScreenshotHandler:** Attaches and normalizes screenshots.

10. PERFORMANCE BENCHMARK

10,000+
PROBLEM FOLDERS

< 2.3s
SCAN TIME

Benchmark detail: Scalability test for Repository Intelligence scanning a LeetCode structure, showing $O(N)$ linear scaling.

2. Why did you build it?

To remove repetitive manual work involved in organizing, documenting, and publishing coding solutions.

4. Why modular architecture?

Each component has one responsibility, making the system easier to maintain and modify.

6. What happens if a stage fails?

GitGo handles the failure and rolls back previous changes so there is no partial publish.

8. How does GitGo support Pull Requests?

It creates a branch, commits the solution, pushes the branch, and generates a PR.

10. Why use TypeScript?

Because GitGo is a VS Code extension and TypeScript works naturally with the VS Code Extension API and Node.js.

12. Explain the 10,000+ / 2.3 sec claim.

It represents the scan time of the Repository Intelligence module, scaling linearly with $O(N)$ file-system queries.